

The Unified Sovereign-Tenant Cloud: Linear Shared-Nothing Scaling via Per-Tenant SQLite and In-Process Composition

Hanzo AI Research
Hanzo Industries Inc
research@hanzo.ai

Measurement pass: 2026-07-08 (host evo, AMD Ryzen AI Max+ 395, Go 1.26.4)

Abstract

Hanzo Cloud runs the entire control- and data-plane of a multi-tenant SaaS as *one* ZAP-native Go binary (56 in-process subsystems, one runtime) over a *shared-nothing* storage substrate: every tenant — org, app, project, or user — is its own embedded SQLite file, replicated to durable object storage as per-tenant post-quantum-encrypted LTX segments, served by a single pinned writer per file, with only the *active* tenants held open in a bounded per-pod LRU. This paper asks one question and answers it with measurement, not marketing: *what does a resident tenant actually cost, and how does the design scale?* We measure the marginal resident-set size (RSS) of opening $N \in \{1, 10, 100, 1000\}$ real base-schema tenant databases through the exact production open path, on the production driver. The answer: **an open, actively-served tenant costs ≈ 0.25 MB of RAM** (251 KB measured incremental slope; cgo/SQLite, warm workload) — roughly $8\times$ below the 2 MB figure the design was previously budgeted against — and an *idle* tenant costs essentially zero live RAM (it is a file on disk plus an encrypted object). The tuned `cache_size` cap is a *ceiling*, not a footprint: for realistic tenants the working set, not the cap, sets the cost. From this one measured constant we build a capacity model: at 1% peak concurrency, **one billion registered tenants** is served by ≈ 103 32 GB nodes (≈ 2.5 TB cluster RAM); at 0.1% concurrency, 11 nodes. Memory scales with *concurrency*, not with registrations. We contrast this against the default industry shape — shared Postgres behind a microservice mesh — and quantify the penalties it pays that the unified design does not: a global write bottleneck, a hard connection ceiling, and an N -runtime plus sidecar memory tax we measure at $11\text{--}32\times$ the unified binary’s footprint. We are explicit about the limits: writer-pinning failover, hot-tenant skew, cross-tenant queries, and the routing directory are real costs, discussed honestly in §7. Every number labelled `[measured]` is measured on the named host; `[modeled]` and `[cited]` numbers are labelled as such; none are invented.

1 Introduction

The dominant shape of a multi-tenant SaaS backend is a fleet of stateless microservices in front of a shared relational database (typically Postgres), with tenancy expressed as a `tenant_id` column and enforced in application code or row-level security. That shape has two structural costs that grow with the business. First, *writes serialize*: every tenant’s mutations funnel through one primary’s write-ahead log, so the write path is a single global resource no matter how many read replicas are added. Second, *runtimes multiply*: each microservice is a full process — its own language runtime, heap, connection pool, and (in a service mesh) a sidecar proxy — so the fixed memory cost of the platform is paid N times over, before a single byte of tenant data is stored.

Hanzo Cloud takes the opposite position on both axes, and this paper is the measured defense of that position. On composition: the 56 subsystems (IAM, KMS, Base, Commerce, AI, o11y, VFS, MQ, tasks, audit, ...) are linked into *one* binary and communicate in-process over a ZAP-native transport, with the same symbols exposed as ZAP-RPC only where process isolation is a compliance requirement (payments/vault, PCI CDE) [1, 3, 4]. The unified binary’s construction is developed in the companion architecture paper [2]; this paper is its measured memory and scaling counterpart. On storage: there is no shared database. Each tenant is a sovereign, independently encrypted SQLite file [5]; the write path for tenant t is a single writer pinned to t , so writes are *embarrassingly sharded* and there is no global lock, no distributed transaction, and no per-write consensus round. The claim is that this is simultaneously the more *efficient* design (less RAM per tenant, far less fixed overhead) and the more *scalable* one (node count grows linearly in active tenants, with no shared bottleneck to saturate).

Claims about efficiency and scale are cheap; the contribution here is to make them measurable. We (i) establish the real per-active-tenant memory constant by direct measurement through the production open path (§3); (ii) turn that constant into a capacity model from 100 to 10^9 registered tenants (§4); (iii) quantify the memory gap against the shared-Postgres-plus-mesh alternative (§5); and (iv) state the scaling claims as propositions grounded in the measurement, with an honest accounting of where the design pays (§6–§7).

2 Architecture

One binary, 56 in-process subsystems. Per HIP-0106 [1], every Hanzo Go service exposes a top-level `Mount(app, deps)` symbol and is compiled into a single `cloud` binary rather than deployed as an independent service. Subsystems consume each other through Go interfaces that resolve to *in-process* implementations by default (the in-process plugin/composition model of HIP-0116 [3]), falling back to ZAP-RPC only for the small set of subsystems that must stay in a separate process for compliance isolation. HIP-0125 [4] eliminates gRPC from the transport entirely in favor of ZAP-native framing, so there is no per-subsystem gRPC server, no protobuf reflection service, and no sidecar. The consequence for this paper is that the platform’s *fixed* memory cost — one Go runtime, one scheduler, one set of shared caches and connection machinery — is paid *once* per node, not once per subsystem.

The sovereign tenant: one encrypted SQLite file each. Tenancy is a composable hierarchy under one org: an org-wide database, a per-app database, a per-app-per-project database, and a per-user database [5]. Each lands on disk at `{DataDir}/orgs/{orgSlug}/{service}.db` and is opened through exactly one code path — the canonical Hanzo SQLite driver with a fixed pragma battery (§3). There is no shared schema and no cross-tenant table: tenant isolation is a *filesystem* boundary, not a `WHERE tenant_id = ?` predicate, which removes an entire class of tenant-bleed bugs by construction [8].

The pinned single writer. A single tenant is served by exactly one pod at a time via gateway-side sticky affinity (consistent hash on `X-Org-Id`). Because SQLite is a single-writer store and each tenant maps to one file served by one pod, the writer for tenant t is globally unique without any coordination protocol. Reads may be served by read-only replica pods that hydrate from the replication stream and fail *closed* on any mutating verb, so correctness never rides on routing being perfect. This is the mechanism behind the linear-scaling claim: distinct tenants share no write path, so adding tenants adds independent writers rather than contending for one.

Durability: encrypted replication, not a shared database. Per HIP-0302 [5], each tenant’s SQLite file is continuously replicated as LTX segments through `encreplica`, which post-quantum-age-encrypts every segment with a *per-tenant* key before it touches durable storage (the HIP-0107 replication substrate [6]; local FS for dev, S3 or Hanzo VFS in production [7]). The storage and encryption architecture is developed in the companion post-quantum object-store paper [9]; this paper concerns its memory and scaling behavior. At-rest encryption therefore lives on the replication path, not in the resident page cache: the open handle is plaintext SQLite (fast), and the durable copy is ciphertext keyed to the tenant. This distinction matters for the memory model — the crypto cost is transient (a buffer during flush), not a standing per-connection tax.

Active-set residency. Only *active* tenants are held open. The per-pod store keeps open SQLite handles in a bounded LRU (default cap 1000 handles/pod, tunable) with a 5-minute idle reaper; on eviction it checkpoints the WAL, closes the handle, and flushes the encrypted bytes to the object store. A cold tenant occupies no RAM — it is an encrypted object plus, at most, a file in a local cache. This is the architectural reason the resident memory of a node tracks *concurrency* rather than *registrations*, which §3 then quantifies.

3 Benchmark: the real per-tenant memory constant

Question. What is the marginal resident memory of one open, actively-served tenant database, measured through the exact production open path, on the driver the production binary actually links?

Method. We open tenant databases with the identical call the production store uses,

```
dbx.Open("sqlite", sqlite.PragmaDSN(path, sqlite.DefaultPragmas)),
```

where `DefaultPragmas` is the canonical Hanzo battery: `busy_timeout=10000`, `journal_mode=WAL`, `synchronous=NORMAL`, `foreign_keys=ON`, `temp_store=MEMORY`, `cache_size=-32000` (a 32 MiB page-cache *cap* per connection). The seed is the checked-in base test database (`base/tests/data/data.db`, 344,064 B data + 49,152 B auxiliary ≈ 0.38 MB), a realistic small tenant carrying the base system collections plus a little data. For $N \in \{1, 10, 100, 1000\}$ we copy the seed to a fresh path, open it, run a workload, and keep the handle resident; at each checkpoint we force a Go GC and `FreeOSMemory` and read `VmRSS` from `/proc/self/status`. We report the *incremental slope* $\Delta\text{RSS}/\Delta N$ between checkpoints, which cancels the fixed process baseline and yields the true per-tenant constant. RSS (not Go heap statistics) is the honest measure because the cgo SQLite page cache is allocated in the C heap, invisible to Go’s allocator. Two workloads: *cold* (open + a single `SELECT 1`) and *warm* (touch every table’s pages via `count(*)` then perform a small write, materializing the WAL, the `-shm` index, and the pinned writer connection). The production driver is `cgo/mattn` (the `cloud` binary links it as the active `"sqlite"` driver); we also measure the pure-Go `modernnc` backend used on `!cgo` builds. Host: `evo`, AMD Ryzen AI Max+ 395 (32 cores), 62.4 GB, Linux 6.17.0-35, Go 1.26.4.

Result: ≈ 0.25 MB per active tenant. Table 1 gives the measured constants. On the production driver, a warm resident tenant costs a measured **251 KB** (≈ 0.245 MB) at the asymptotic ($100 \rightarrow 1000$) slope; a cold-but-open tenant costs 235 KB. The pure-Go backend is $\approx 1.5\times$ heavier at 392 KB. The single most important finding is in the `cache_size` column: sweeping the cap across $\{-256, -1024, -2000, -32000\}$ (256 KiB to 32 MiB) moves the small-tenant constant by *less than*

0.2% (251.3–251.7 KB). The 32 MiB cap is never approached, because the tenant’s working set (≈ 0.25 MB) — not the cap — sets the footprint. The four cache settings are in effect four independent repeats at equal working set, and their agreement to within 0.4 KB is our repeatability evidence.

Table 1: Measured marginal RSS per open, resident tenant (base-schema seed, 0.38 MB on disk). Incremental slope is $\Delta\text{RSS}/\Delta N$ over $100 \rightarrow 1000$ handles; baseline (runtime + driver, zero tenants) is 7.0–7.2 MB (cgo), 6.3–6.5 MB (modernc). [\[measured\]](#)

Driver	Workload	cache_size	Per-tenant RSS
mattn / SQLite (cgo, <i>production</i>)	warm	−32000 (32 MiB)	251.3 KB (0.245 MB)
mattn / SQLite (cgo, <i>production</i>)	warm	−2000 (2 MiB)	251.4 KB
mattn / SQLite (cgo, <i>production</i>)	warm	−1024 (1 MiB)	251.3 KB
mattn / SQLite (cgo, <i>production</i>)	warm	−256 (256 KiB)	251.4 KB
mattn / SQLite (cgo, <i>production</i>)	cold	−32000	235.5 KB (0.230 MB)
modernc (pure-Go, !cgo)	warm	−32000	392.0 KB (0.383 MB)

The density knob is real — for *hot, large* tenants only. That `cache_size` is inert for small tenants is not the whole story. We re-ran the sweep against a synthetic 25 MB tenant under a full-scan warm workload (Table 2). Here the cap binds: at −32000 a fully-scanned hot tenant fills its page cache to the working set and costs 25.5 MB; at −1024 it is bounded to 1.18 MB; at −256 to 0.41 MB. So the cap is a per-tenant memory *ceiling* operators can dial to trade page-cache hit rate for density. The realistic operating point is Table 1 (working-set-bound, ≈ 0.25 MB); Table 2 is the worst case (analytic tenant scanning its whole database every request) and shows it remains bounded and tunable rather than unbounded.

Table 2: Density knob: marginal RSS for a *hot* 25 MB tenant under a full-table-scan workload, as a function of the `cache_size` cap. [\[measured\]](#)

cache_size cap	Per-tenant RSS	Effect
−32000 (32 MiB)	25.5 MB	cache fills to working set (cap not reached)
−1024 (1 MiB)	1.18 MB	bounded to ≈ 1 MiB regardless of DB size
−256 (256 KiB)	0.41 MB	bounded to ≈ 256 KiB + fixed overhead

Idle tenants cost ≈ 0 . We opened and exercised 1000 tenants, then closed every handle. On the pure-Go backend — whose memory frees to the Go heap, which `FreeOSMemory` returns to the OS — RSS fell from 392 MB back to **11.2 MB**, i.e. to within 4.9 MB of baseline over 1000 closed tenants (≈ 5 KB/tenant of retained runtime bookkeeping). This is the direct measurement behind “an idle tenant is free”: closing a handle releases essentially all of its live memory. On the cgo backend RSS settled at 221 MB after close; that residual is *not* live memory but glibc’s retention of freed C arenas (the allocator does not `munmap` them back to the kernel). We confirmed it is reused, not leaked: closing 1000 tenants and opening 1000 fresh ones grew RSS by only ≈ 110 MB on top of the 222 MB retained (vs. 247 MB for a cold start), so the freed arenas absorb subsequent opens. The operational takeaway: a pod’s steady-state RSS tracks its *high-water active set*, and idle registrations contribute no live pages.

On-disk and base-node constants. A realistic tenant occupies ≈ 0.38 MB on disk (measured seed), which at rest lives as per-tenant-encrypted LTX in object storage [7, 5], not on node-local disk. The fixed per-node cost — the `cloud` binary with all 56 subsystems and one Go runtime — is a working-set RSS of **154–225 MB** from prior production measurement [10]; we use 190 MB as the point estimate in the model below and note it is [cited] (the cluster was not reachable from the measurement host on this pass to re-sample it live). For comparison, a *bare* Go HTTP service (net/http, one handler, no framework) measured [measured] 7.2 MB on the same host — the floor a per-service split cannot go below, used in §5.

4 The capacity model

The measured constant makes the fleet size a closed-form function of registrations and concurrency. Let R be registered tenants, c the peak concurrency fraction, so active tenants $A = cR$. Let m be the measured per-active-tenant RAM (MB), b the per-node base RSS (GB), and U the usable RAM per node (GB) after reserving for the OS, kubelet, and burst headroom. Then

$$\text{nodes} = \max\left(3, \left\lceil \frac{A \cdot m}{(U - b) \cdot 1024} \right\rceil\right), \quad \text{cluster RAM} = \text{nodes} \cdot b + \frac{A \cdot m}{1024} \text{ GB}. \quad (1)$$

We use 32 GB nodes with $U = 24$ GB (75% usable), $b = 0.19$ GB, and the measured $m = 0.25$ MB, giving a per-node density of $\approx 97,500$ active tenants; we also report the conservative $m = 1.0$ MB envelope (larger/hotter tenants, multiple resident connections, extra headroom), which yields $\approx 24,400$ active/node. Disk is $R \times 0.38$ MB total, object-tiered. Table 3 gives the model at 1% peak concurrency; Table 4 shows concurrency sensitivity at the extremes.

Table 3: Capacity at **1% peak concurrency**, 32 GB nodes, from Eq. 1 with the [measured] $m = 0.25$ MB constant (and the [modeled] $m = 1.0$ MB conservative envelope). “Cluster RAM” is consumed RAM per Eq. 1, not provisioned. Node counts are floored at 3 for HA.

Registered R	Active @1%	Nodes ($m=0.25$)	Cluster RAM	Nodes ($m=1.0$)	Disk (total, tiered)
100	1	3	0.57 GB	3	40 MB
1,000	10	3	0.57 GB	3	400 MB
10,000	100	3	0.59 GB	3	4 GB
100,000	1,000	3	0.81 GB	3	40 GB
1,000,000	10,000	3	3.01 GB	3	400 GB
10,000,000	100,000	3	24.98 GB	5	4 TB
100,000,000	1,000,000	11	246.2 GB	42	40 TB
1,000,000,000	10,000,000	103	2.46 TB	411	400 TB

Table 4: Concurrency sensitivity for $R = 10^9$ registered tenants ($m = 0.25$ MB, 32 GB nodes). Memory scales with *concurrency*, not registrations: a 50× swing in peak concurrency is a 50× swing in fleet size, while R is held fixed. [measured]-grounded.

Peak concurrency	Active tenants	Nodes	Cluster RAM
0.1%	1,000,000	11	246 GB
1.0%	10,000,000	103	2.46 TB
5.0%	50,000,000	513	12.3 TB

The shape of Table 3 is the thesis in numbers. Up to 10 million registered tenants at 1% concurrency, the fleet is the 3-node HA floor — the platform is base-RSS-bound, not tenant-bound, and the tenants are “free” relative to the fixed cost of running at all. Only past $\sim 10^8$ registrations does tenant memory dominate, and there it grows *linearly*: 10^9 tenants is 103 nodes, 10^{10} would be ~ 1030 . There is no knee, no bottleneck tier, and no rewrite between the 3-node and the 103-node regime — the same binary and the same per-tenant file, more of them.

5 Comparative analysis: shared Postgres + microservice mesh

We compare against the default industry shape on the three axes where the unified design’s structure diverges. The comparison is deliberately conservative: where we estimate the alternative we anchor to a measured floor and label the estimate [modeled].

(1) The write path: sharded vs. global. In shared Postgres, every tenant’s writes commit through one primary’s WAL. Read replicas scale reads but not writes; the write ceiling is one node’s fsync throughput, and it is *shared* across all tenants, so a write-heavy tenant degrades every other tenant’s write latency. Scaling writes past one primary requires sharding (Citus, Vitess-style) or multi-primary consensus (Raft/Paxos per write) — either a re-architecture or a per-commit coordination round. In the unified design the write path for tenant t is a private SQLite writer; N tenants are N independent writers with *no shared lock and no consensus round*. Aggregate write throughput is the sum of per-tenant throughputs and grows linearly with nodes. The cost paid for this is that a single tenant’s writes are bounded by one file’s single-writer throughput (§7); the benefit is that there is no global write resource to saturate.

(2) The connection ceiling. A Postgres primary allocates a backend process per connection (~ 5 – 10 MB each), so practical `max_connections` is in the hundreds; thousands of tenant-affine connections require a pooler (PgBouncer) and still contend for the one primary. Per-tenant SQLite has no connection ceiling: a “connection” is an in-process handle into a memory-mapped file, and the active-set LRU (§2) bounds the resident count per pod directly. There is no network round trip and no server-side backend process on the data path at all.

(3) The N -runtime + sidecar memory tax. This is the axis we can quantify most directly. The unified binary runs all 56 subsystems in one Go runtime at a measured/cited base of 190 MB; at 3 replicas for HA that is 570 MB of control-plane RAM before tenant data. A microservice mesh runs each subsystem as its own process with its own runtime heap and, under a service mesh, a sidecar proxy. Even at the *measured floor* of a bare Go service (7.2 MB [measured]) plus a single Istio/Envoy sidecar (≈ 50 MB [cited] [12]) at 2 replicas, 56 subsystems cost 6.4 GB; at a realistic 60 MB/service ([modeled]: framework + OTEL SDK + DB pool + business logic) plus 50 MB sidecar, 12.3 GB. Table 5 gives the multiples: the mesh spends 11–32 $\times$ the unified binary’s fixed memory, and adds one network hop (plus TLS and serialization) to every inter-subsystem call that the unified design makes as an in-process function call.

Composed with the capacity model, the gap compounds: the mesh’s 6–21 GB fixed tax is paid on *every node*, so at the 3-node HA floor — where the unified platform serves up to 10^7 tenants in <1 GB of control-plane RAM (Table 3) — the mesh alternative already needs 19–64 GB just to exist.

Table 5: Fixed control-plane memory: unified binary vs. 56-subsystem microservice mesh, before any tenant data. Unified base 190 MB is [measured]/[cited]; bare-Go 7.2 MB is [measured]; per-service framework RSS and sidecar RSS are [modeled]/[cited]. Multiple is mesh $\div$ unified.

Deployment	Per-service	Fixed RAM	$\times$ unified (3-rep, 570 MB)
Unified binary, 3 replicas	–	570 MB	1.0 $\times$
Mesh, floor (7.2 MB svc + 50 MB sidecar, 2 rep)	57.2 MB	6.4 GB	11.2 $\times$
Mesh, realistic (60 MB svc + 50 MB sidecar, 2 rep)	110 MB	12.3 GB	21.6 $\times$
Mesh, heavy (120 MB svc + 70 MB sidecar, 2 rep)	190 MB	21.3 GB	37.3 $\times$

6 Claims, grounded in the measurement

We state the design’s guarantees as propositions, each tied to a measured or structural fact rather than an assertion.

Proposition 1 (memory $\propto$ concurrency, not registrations). Resident memory is nodes $\cdot b + A \cdot m$ with $A = cR$ (Eq. 1). Because an idle tenant contributes ≈ 0 live RAM (measured: 1000 closed handles return to within 4.9 MB of baseline, §3) and only the active-set LRU is resident, RAM is a function of A , and R enters only through $A = cR$. Doubling registrations at fixed concurrency does not change resident memory; it changes only object-storage bytes. Table 4 is the empirical shape.

Proposition 2 (linear node scaling from shared-nothing writers). Distinct tenants share no write path: each is a private single-writer SQLite file pinned to one pod (§2). There is no global lock, no distributed transaction, and no per-write consensus, so adding a node adds independent write and read capacity with no coordination overhead that grows with the fleet. Fleet size is therefore $\Theta(A)$ — 103 nodes at 10^7 active, ~ 1030 at 10^8 — with a constant factor set by the measured m . This is linear, not merely sublinear-with-a-bottleneck.

Proposition 3 (the one-binary memory advantage is multiplicative). Composing S subsystems into one runtime pays the fixed runtime + sidecar cost once rather than S times. Measured against a 56-subsystem mesh, the fixed-memory ratio is 11–32 $\times$ (Table 5), and it is paid per node, so it multiplies the per-node base b in Eq. 1. Lowering b is what keeps small deployments in the sub-gigabyte regime in Table 3.

7 Limitations

The rigor is the persuasion, so the costs are stated as plainly as the benefits.

- **Writer-pinning failover.** One writer per tenant means a pod loss requires re-pinning that tenant to a new pod and replaying the tail of the encrypted replication stream before writes resume. HIP-0302’s consistency model is at-most-once under rebalance: a short single-writer overlap window can exist during an HPA move, and operators must drain before scaling. There is a bounded write-unavailability window per affected tenant on failover; the CAS (ETag If-Match) follow-up is required for generation-checked writes across a rebalance.
- **Hot-tenant skew.** Capacity is modeled on an *average* tenant. One tenant that is simultaneously large and hot can, at the default 32 MiB cap, cost up to 25.5 MB resident (Table 2)

and is bounded to one pod’s single-writer throughput. The mitigations — a smaller per-tenant `cache_size`, splitting a heavy org across the app/project/user hierarchy, and per-tenant rate ceilings — are real levers but they are operational work, not free. The model’s m should be chosen for the p99 tenant, not the mean, in skewed populations.

- **Cross-tenant queries.** A filesystem isolation boundary is exactly the wrong shape for a query that must span tenants (fleet-wide analytics, admin search, billing rollups). These require fan-out plus aggregation, or a separate columnar sink fed from the replication stream — neither is a single SQL statement. The design optimizes the 99% per-tenant path at the cost of the cross-tenant path.
- **The routing directory.** Sticky single-writer routing needs a consistent, durable map from tenant to pod. That directory is itself a piece of shared state that must be highly available and consistent; it is small and changes slowly, but it is a coordination point the pure shared-nothing story would prefer not to have.
- **Measurement scope.** The per-tenant constant is measured on one host, one seed schema, single-stream warm/cold workloads, with a bounded 2-connection pool. It does not sweep concurrent query load per tenant (which materializes more connections, each with its own page cache), nor multi-hour steady-state fragmentation. The 190 MB base RSS is [cited] from prior production sampling, not re-measured on this pass. These are the honest edges of the numbers.

8 Conclusion

The unified sovereign-tenant design is efficient and scalable for the same structural reason: it refuses to share the two resources that normally become bottlenecks. It does not share a write path — each tenant is a private single-writer SQLite file — so writes shard perfectly and node count grows linearly in active tenants. It does not share N runtimes — 56 subsystems live in one binary — so the fixed platform cost is paid once, at 11–32 $\times$ less memory than the equivalent mesh. The measured per-active-tenant cost is ≈ 0.25 MB, an idle tenant is free, and the `cache_size` cap is a tunable ceiling rather than a footprint. The capacity model that follows from that one constant puts a billion registered tenants on ~ 100 commodity nodes at 1% concurrency, and on 11 at 0.1% — with no bottleneck tier and no rewrite between the small and the large regime. The design is not free of cost: writer failover, hot-tenant skew, and cross-tenant queries are genuine prices, stated in §7. But on the axis it is built for — serving an enormous number of independent tenants cheaply and scaling their concurrency linearly — the measurement supports the architecture.

A Reproducibility

Host. evo: AMD Ryzen AI Max+ 395 (32 cores), 62.4 GB RAM, Linux 6.17.0-35, Go 1.26.4, glibc malloc, TeX Live 2023.

Open path. `github.com/hanzoai/dbx v1.16.0 + github.com/hanzoai/sqlite v0.2.1;` production driver `cgo/mattn (go-sqlite3 v1.14.47);` pure-Go control `modernc.org/sqlite v1.51.0`. Pragmas exactly `sqlite.DefaultPragmas: busy_timeout=10000, journal_mode=WAL, synchronous=NORMAL, foreign_keys=ON, temp_store=MEMORY, cache_size=-32000`.

Seed. `base/tests/data/data.db` (344,064 B) + `auxiliary.db` (49,152 B), the checked-in base-schema test database.

Procedure. Copy seed to a fresh path per tenant; open through the production path; run the

warm workload (`count(*)` over every non-`sqlite_` table, then `CREATE TABLE IF NOT EXISTS` + five `INSERTs`) or the cold workload (`SELECT 1`); at each of $N \in \{1, 10, 100, 1000\}$ force `runtime.GC()` + `debug.FreeOSMemory()` and read `VmRSS` from `/proc/self/status`. Report the $100 \rightarrow 1000$ incremental slope. Idle test: close all handles, re-measure; reuse test: close all, open 1000 fresh, re-measure. Large-tenant knob: inflate the seed to 25 MB (60,000 padded rows) and repeat the `cache_size` sweep under a full-scan warm workload.

Capacity model. Eq. 1 with $U = 24$ GB, $b = 0.19$ GB, $m \in \{0.25, 1.0\}$ MB, 32 GB nodes, HA floor 3; disk $R \times 0.38$ MB object-tiered.

References

- [1] Hanzo AI. *HIP-0106: Cloud — Unified Hanzo Binary*. <https://github.com/hanzoai/HIPs>.
- [2] Z. Kelling. *The Unified Cloud Binary: Collapsing a Microservices Control Plane into One Go Process*. Hanzo Industries white paper, 2026 (companion architecture paper; `cloud-unified-binary`).
- [3] Hanzo AI. *HIP-0116: Plugin VM / In-Process Composition Model*. <https://github.com/hanzoai/HIPs>.
- [4] Hanzo AI. *HIP-0125: ZAP-Native Transport & gRPC Elimination* (renumbered from HIP-0116 to avoid collision with the plugin-vm model). <https://github.com/hanzoai/HIPs>.
- [5] Hanzo AI. *HIP-0302: Hanzo Replicate — Encrypted SQLite Durability for Base Services*. <https://github.com/hanzoai/HIPs>.
- [6] Hanzo AI. *HIP-0107: Replication and Durability Substrate*. <https://github.com/hanzoai/HIPs>.
- [7] Hanzo AI. *HIP-0032: Object Storage Standard*. <https://github.com/hanzoai/HIPs>.
- [8] Hanzo AI. *HIP-0118: SuperAdmin and Tenant Isolation Model*. <https://github.com/hanzoai/HIPs>.
- [9] Z. Kelling. *A Post-Quantum Object Store: Per-Tenant SQLite and a Dropbox-Class Filesystem over One Encrypted Block Substrate*. Hanzo Industries white paper, 2026 (companion; `hanzo-pq-object-store`).
- [10] Hanzo AI. *Edge LLM Inference Across Commodity Accelerators* (companion white paper; source of the prior 154–225 MB cloud working-set measurement).
- [11] SQLite. *PRAGMA cache_size and Write-Ahead Logging*. https://www.sqlite.org/pragma.html#pragma_cache_size, <https://www.sqlite.org/wal.html>.
- [12] Istio. *Sidecar resource footprint* (Envoy proxy memory, per-proxy). <https://istio.io/latest/docs/ops/deployment/performance-and-scalability/>.